Cahier des charges — FlowTrack

Version : 1.0 — MVP

1. Présentation du projet

1.1 Nom

FlowTrack

1.2 Nature du projet

FlowTrack est une plateforme de **gestion intelligente des tâches, des objectifs et de la collaboration**, accessible depuis le Web et installable comme **Progressive Web App (PWA)**.

L'application combine :

- Gestion des tâches
- Gestion des objectifs
- Planification
- Collaboration
- Suivi de progression
- Gamification
- Intelligence artificielle
- Notifications
- Fonctionnement hors ligne

1.3 Vision

FlowTrack doit permettre à une personne ou à une équipe de répondre rapidement à trois questions :

- **Que dois-je faire ?**
- **Qu'est-ce qui est réellement prioritaire ?**
- **Est-ce que je progresse ?**

L'application doit transformer une liste potentiellement désordonnée de tâches en une **vision claire et actionnable du travail à accomplir**.

---

2. Problématique

Les outils traditionnels de gestion des tâches permettent généralement de stocker des tâches, mais ne résolvent pas nécessairement les problèmes suivants :

- Difficulté à déterminer les priorités
- Accumulation de tâches
- Manque de visibilité sur la progression
- Difficulté à transformer un objectif en actions concrètes
- Manque de motivation
- Difficulté à organiser le travail collectif
- Fragmentation des informations
- Difficulté à maintenir une organisation lorsqu'on perd sa connexion Internet

FlowTrack doit répondre à ces problèmes en associant **organisation + intelligence + collaboration + motivation**.

---

3. Objectifs

3.1 Objectif principal

Créer une application conviviale permettant à l'utilisateur de **centraliser, prioriser, planifier et suivre ses tâches et objectifs**, individuellement ou en équipe.

3.2 Objectifs secondaires

FlowTrack devra permettre de :

- Réduire la charge cognitive liée à l'organisation

- Identifier automatiquement les tâches importantes
- Améliorer la visibilité sur l'avancement
- Faciliter le travail en équipe
- Transformer les objectifs en actions
- Encourager la régularité
- Fournir des statistiques utiles
- Fonctionner même sans connexion Internet
- Utiliser l'IA comme véritable copilote de productivité

---

4. Public cible

FlowTrack vise principalement le **grand public**.

Les utilisateurs pourront notamment être :

- Étudiants
- Développeurs
- Designers
- Entrepreneurs
- Freelances
- Employés
- Responsables d'équipe
- Petites équipes
- Utilisateurs souhaitant simplement mieux organiser leur quotidien

L'application devra rester suffisamment simple pour être utilisée par un débutant tout en étant suffisamment puissante pour accompagner des utilisateurs avancés.

---

5. Concept général

Le modèle fonctionnel principal sera :

Utilisateur

```
    |
    ├── Workspace personnel
    |     └── Projets
    |           └── Tâches
    |
    └── Workspace collaboratif
          ├── Membres
          ├── Rôles
          ├── Projets
          |     └── Tâches
          └── Activité
```

Les objectifs pourront être associés à des tâches :

Objectif

```
  |
  ├── Milestone
  |
  ├── Tâche
  ├── Tâche
  └── Tâche
```

La progression de l'objectif sera calculée à partir de l'état des éléments qui lui sont associés.

---

6. Gestion des utilisateurs

6.1 Création de compte

L'utilisateur pourra créer un compte avec :

- Adresse email
- Mot de passe

Le système devra prévoir :

- Validation de l'adresse email
- Connexion
- Déconnexion
- Changement de mot de passe
- Récupération de mot de passe
- Suppression du compte

6.2 Profil

Chaque utilisateur disposera d'un profil comprenant notamment :

- Nom
- Avatar
- Adresse email
- Préférences
- Statistiques
- Niveau XP
- Badges
- Streak

- Préférences de notifications

---

7. Workspaces

Le Workspace constituera l'espace de travail principal.

Un utilisateur pourra posséder ou rejoindre plusieurs Workspaces.

Exemple :

Eric

│

├── Personnel

│

├── Portfolio

│

└── FlowTrack Team

Chaque Workspace pourra contenir :

- Des membres
- Des projets
- Des tâches
- Des objectifs
- Des paramètres
- Des statistiques

---

8. Rôles

Le système devra gérer plusieurs niveaux de permissions.

Owner

Le propriétaire pourra :

- Gérer le Workspace
- Inviter des utilisateurs
- Modifier les permissions
- Supprimer le Workspace
- Gérer les projets

Administrator

L'administrateur pourra :

- Gérer les membres
- Gérer les projets
- Gérer les tâches
- Gérer certains paramètres

Member

Le membre pourra :

- Consulter les projets autorisés
- Créer des tâches
- Modifier ses tâches
- Collaborer
- Participer aux challenges

Guest

Le guest aura des permissions limitées.

Le système de permissions devra être extensible afin de pouvoir ajouter ultérieurement des rôles personnalisés.

---

9. Gestion des tâches

Une tâche FlowTrack pourra contenir l'ensemble des informations nécessaires à une gestion avancée.

Informations principales

- Titre
- Description
- Statut
- Priorité
- Importance
- Urgence
- Date de création
- Échéance
- Heure
- Catégorie
- Projet
- Tags
- Responsable
- Collaborateurs
- Sous-tâches
- Estimation de durée
- Récurrence
- Pièces jointes
- Notes

Statuts

À faire

↓

En cours

↓

En pause

↓

Terminée

↓

Archivée

Une tâche pourra également être déplacée directement vers certains états lorsque cela est nécessaire.

---

10. Priorisation intelligente

La priorité ne devra pas uniquement être une valeur saisie manuellement.

FlowTrack combinera :

- Priorité utilisateur
- Urgence
- Importance
- Échéance
- Contexte
- Charge de travail
- Progression
- Recommandations IA

L'objectif est de produire un **score de priorité** permettant de classer les tâches.

Exemple conceptuel :

Score priorité

|

├— Urgence

├— Importance

├— Échéance

├— Impact

└— Contexte

↓

Priorité recommandée

L'utilisateur pourra toujours modifier la recommandation.

---

11. Vue « Aujourd'hui »

La vue **Aujourd'hui** constituera l'une des fonctionnalités centrales de FlowTrack.

Elle devra répondre immédiatement à :

« Que dois-je faire maintenant ? »

Elle pourra afficher :

- Tâches prioritaires
- Tâches arrivant à échéance
- Tâches en retard
- Objectifs actifs
- Progression du jour

- Planning recommandé par l'IA
- Streak
- Recommandations

---

12. Planning intelligent

L'IA pourra analyser les tâches disponibles et proposer une organisation quotidienne.

Exemple :

Votre journée contient 6 tâches estimées à 5 h 20.

09 : 00 — Finaliser la présentation

10 : 30 — Réunion équipe

11 : 30 — Corriger les bugs prioritaires

14 : 00 — Travail sur l'objectif JavaScript

16 : 00 — Révision des tâches restantes

L'utilisateur pourra :

- Accepter
- Modifier
- Refuser
- Régénérer le planning

---

13. Dashboard personnalisable

Le dashboard devra être **modulaire**.

L'utilisateur pourra choisir les widgets qu'il souhaite afficher.

Exemples :

- Aujourd'hui
- Priorités
- Progression
- Objectifs
- Statistiques
- Streak
- XP
- Badges
- Challenges
- Activité d'équipe
- Classement
- Recommandations IA

À terme, l'utilisateur pourra également :

- Déplacer les widgets
- Modifier leur taille
- Masquer certains widgets
- Sauvegarder plusieurs configurations

---

14. Gestion des objectifs

Un objectif pourra être défini avec :

- Titre
- Description
- Échéance

- Priorité
- Catégorie
- Progression
- Tâches associées
- Milestone
- Statistiques

Exemple :

Objectif

└── Apprendre JavaScript

  │

  ├── Variables & types ✓

  ├── Fonctions ✓

  ├── DOM 70 %

  ├── Async / Await 20 %

  └── Projet final 0 %

L'IA pourra proposer automatiquement une décomposition en étapes.

---

15. Intelligence artificielle

L'IA constitue une **fonctionnalité centrale du produit**.

Elle devra fonctionner comme un **copilote de productivité**.

Elle pourra notamment :

Organisation

- Analyser les tâches

- Détecter les priorités
- Proposer un ordre d'exécution
- Construire un planning

Objectifs

- Transformer un objectif en étapes
- Proposer des tâches
- Détecter les blocages
- Suivre la progression

Analyse

- Analyser les habitudes
- Détecter les périodes de forte productivité
- Identifier les tâches régulièrement repoussées
- Proposer des améliorations

Assistance

L'utilisateur pourra demander :

- « Organise ma journée. »
- « Quelles sont mes trois tâches les plus importantes ? »
- « Pourquoi je prends du retard sur cet objectif ? »
- « Découpe cet objectif en tâches réalisables. »

L'IA devra privilégier des **actions concrètes** plutôt que des réponses génériques.

---

16. Collaboration

La collaboration fera partie du MVP.

Les utilisateurs pourront :

- Créer des tâches partagées
- Assigner des tâches
- Collaborer sur un projet
- Consulter l'activité
- Travailler dans un Workspace commun

L'architecture devra permettre d'ajouter ultérieurement :

- Commentaires
- Mentions
- Fichiers
- Discussions
- Notifications avancées
- Réunions

---

17. Gamification

FlowTrack intégrera un système de progression.

XP

Exemple :

Objectif atteint → +100 XP

Streak → bonus

Challenge terminé → récompense

Badge obtenu → progression

Une tâche en retard ne provoquera **aucune pénalité XP**.

L'objectif est d'encourager la progression plutôt que de sanctionner l'échec.

Badges

Exemples :

- Première tâche terminée
- Première semaine complète
- 10 objectifs terminés
- 30 jours de streak
- Team Player
- Productivity Master

Challenges

Les challenges pourront être :

- Individuels
- D’équipe
- Quotidiens
- Hebdomadaires
- Mensuels

Classement

Le classement sera principalement réalisé **au niveau de l'équipe**.

---

18. Notifications

FlowTrack devra prendre en charge :

Notifications in-app

- Tâche bientôt échue
- Tâche en retard

- Objectif atteint
- Badge obtenu
- Challenge
- Activité d'équipe
- Recommandation IA

Push notifications

La PWA devra pouvoir envoyer des notifications push lorsque le navigateur et la plateforme le permettent.

L'utilisateur pourra contrôler ses préférences.

---

19. Mode hors ligne

FlowTrack devra adopter une approche **offline-first**.

L'utilisateur pourra continuer à :

- Consulter ses données
- Créer des tâches
- Modifier des tâches
- Changer les statuts
- Gérer ses objectifs
- Consulter son dashboard

Lors du retour de la connexion :

Modifications locales

↓

Détection connexion

↓

Synchronisation

↓

API

↓

PostgreSQL

L'architecture devra également prévoir la **gestion des conflits de synchronisation**.

---

20. PWA

La version mobile du MVP sera basée sur une **Progressive Web App**.

Elle devra notamment permettre :

- Installation sur l'appareil
- Fonctionnement hors ligne
- Cache des ressources
- Synchronisation
- Notifications push
- Expérience adaptée aux écrans mobiles

---

21. UX/UI

L'interface devra privilégier :

- Simplicité
- Lisibilité
- Hiérarchie visuelle
- Rapidité

- Feedback immédiat
- Accessibilité
- Responsive design

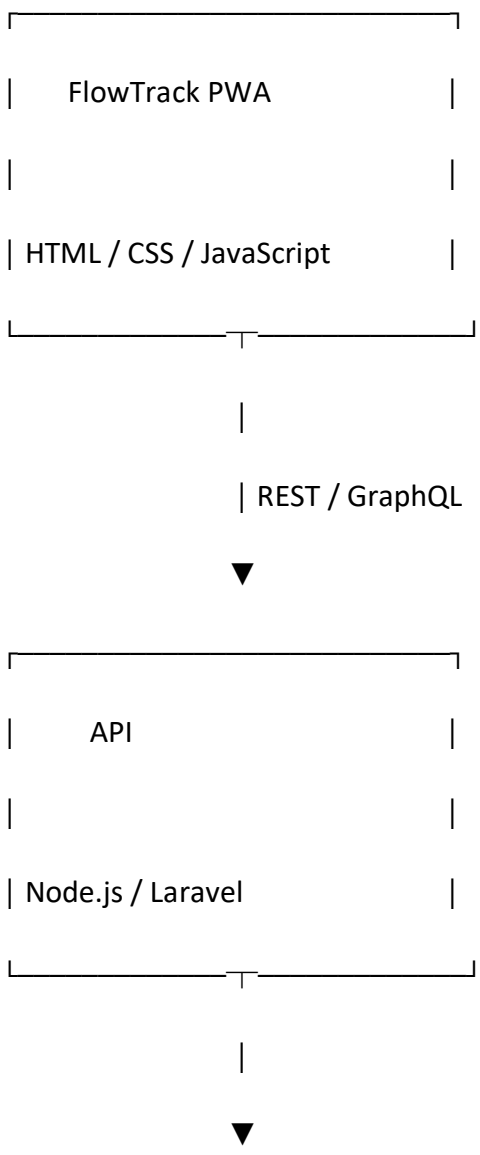
L'application devra éviter de transformer la gestion des tâches en une interface surchargée.

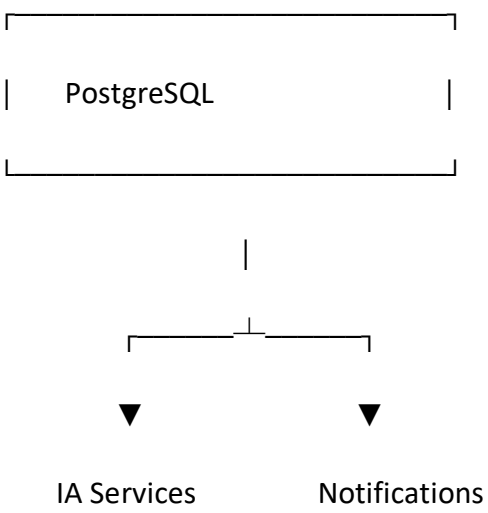
La complexité devra être **progressive** : les fonctions avancées apparaissent lorsque l'utilisateur en a besoin.

---

22. Architecture technique

L'architecture cible sera séparée :





Pour le MVP, je privilégierais **REST** plutôt que GraphQL : l'application possède des ressources assez naturelles (users, workspaces, projects, tasks, goals, etc.), ce qui permettra de garder l'API simple et facilement testable.

Pour le backend, **Node.js** serait mon choix naturel si l'objectif est de rester dans un écosystème JavaScript de bout en bout. **Laravel** reste une excellente alternative si tu souhaites exploiter davantage l'écosystème PHP.

---

23. Modèle de données initial

Les principales entités seront :

User

Workspace

Workspace Member

Role

Project

Task

Subtask

Tag

Goal

Milestone

Attachment

Notification

Challenge

Badge

Reward

UserXP

Streak

Activity

AI Recommendation

Les relations seront définies précisément lors de la conception de la base PostgreSQL.

---

24. API

L'API devra exposer notamment des ressources autour de :

/Auth

/Users

/Workspaces

/Members

/Projects

/Tasks

/Subtasks

/Goals

/Milestones

/Tags

/Notifications

/Challenges

/Badges

/Stats

/AI

Elle devra prévoir :

- Authentification
- Autorisation
- Validation
- Pagination
- Filtrage
- Tri
- Gestion des erreurs
- Logs
- Versionnement de l'API

---

25. Sécurité

Les points suivants seront considérés comme essentiels :

- Mots de passe hashés ;
- Authentification sécurisée
- Gestion des sessions/tokens

- Contrôle des permissions côté serveur
- Validation des entrées
- Protection contre les injections
- Protection XSS
- Protection CSRF lorsque pertinente
- Limitation du nombre de requêtes
- Sécurisation des fichiers
- HTTPS ;
- Journalisation des événements importants
- Principe du moindre privilège

La sécurité ne devra jamais être confiée uniquement au frontend.

---

26. Tests

Le projet devra intégrer une stratégie de tests automatisés.

Tests unitaires

Pour :

- Fonctions métier
- Calcul de priorité
- Calcul de progression
- XP
- Streak
- Règles de permissions

Tests d'intégration

Pour :

- API
- Base de données
- Authentification
- Synchronisation

Tests E2E

Pour les parcours critiques :

→ Inscription

→ Connexion

→ Création Workspace

→ Création projet

→ Création tâche

→ Modification statut

→ Achèvement

→ Progression

Puis les parcours collaboratifs et offline.

---

27. CI/CD

Le projet devra prévoir un pipeline permettant notamment :

Git Push

↓

Lint

↓

Tests unitaires

↓

Tests intégration

↓

Build

↓

Tests E2E

↓

Déploiement

Les environnements pourront être séparés :

Development

↓

Staging

↓

Production

---

28. MVP

Le MVP devra donc inclure :

Core

- Authentification
- Utilisateurs
- Workspaces
- Rôles

- Projets
- Tâches
- Sous-tâches
- Statuts
- Priorités
- Objectifs
- Dashboard
- Kanban
- Vue Aujourd'hui

Collaboration

- Membres
- Assignation
- Tâches partagées
- Activité d'équipe

IA

- Priorisation
- Recommandations
- Décomposition d'objectifs
- Planning quotidien
- Analyse de progression

Motivation

- XP
- Badges

- Streak
- Challenges
- Récompenses
- Classement d'équipe

Infrastructure

- PWA
- Offline
- Synchronisation
- Notifications
- API
- PostgreSQL
- Tests
- CI/CD

---

29. Hors MVP

Certaines fonctionnalités pourront être conservées pour les versions suivantes :

- Applications mobiles natives
- Intégrations Google Calendar
- Slack/Discord/Teams
- Commentaires avancés
- Chat d'équipe
- Automatisations
- Marketplace

- Intégrations tierces
- Fonctionnalités IA autonomes avancées
- Personnalisation poussée des workflows
- Fonctionnalités premium

---

30. Roadmap proposée

Phase 0 — Conception

Objectif : définir précisément le produit.

- Cahier des charges
- Architecture
- Modèle de données
- UX research
- User flows
- Design system
- Prototypes Figma

Phase 1 — Foundation

- Repository
- Architecture frontend
- API
- PostgreSQL
- Authentification
- CI/CD ;
- Environnement de développement

Phase 2 — Core Tasks Management

- Workspaces
- Projets
- Tâches
- Sous-tâches
- Statuts
- Priorités
- Kanban
- Vue Aujourd'hui

Phase 3 — Goals & Dashboard

- Objectifs
- Milestones
- Progression
- Statistiques
- Dashboard personnalisable

Phase 4 — Collaboration

- Membres
- Rôles
- Assignation
- Activité d'équipe
- Classement

Phase 5 — Gamification

- XP

- Badges
- Streaks
- Challenges
- Récompenses

Phase 6 — IA

- Moteur de recommandations
- Priorisation
- Planning
- Décomposition d'objectifs
- Analyse de productivité

Phase 7 — Offline/PWA

- Service Worker
- Cache
- Stockage local
- Synchronisation
- Résolution des conflits
- Push notifications

Phase 8 — QA & Release

- Tests
- Sécurité
- Performance
- Accessibilité
- E2E

- CI/CD
- Staging
- Production

---

31. Critère de réussite du MVP

Le MVP pourra être considéré comme réussi lorsqu'un utilisateur pourra :

Créer un compte → créer ou rejoindre un workspace → créer un projet → définir un objectif → générer des tâches → les prioriser → organiser sa journée avec l'IA → travailler seul ou en équipe → suivre sa progression → gagner de l'XP → continuer à travailler hors ligne → retrouver automatiquement ses données synchronisées.

C'est ce parcours qui doit devenir le **fil conducteur de toute la conception**.